Convenções de Frontend — FrotaManager

Oct 23, 2026 · @Gabrieeeu

Visão geral

O frontend é um repositório com **dois apps independentes** que compartilham componentes, estilos e hooks. Estas convenções valem para os dois; o que é exclusivo do motorista está na seção própria.

App	Público	Porta	Comando	Build
Gestor	Desktop: dashboard, veículos, motoristas, solicitações	5173	`npm run dev:gestor`	`dist/gestor`
Motorista	Mobile-first, PWA online-first	5174	`npm run dev:motorista`	`dist/motorista`

- `npm run dev` sobe os dois ao mesmo tempo.
- Cada app é publicado em seu próprio domínio (ex.: `gestor.…` e `app.…`).
- Stack: React 19, TypeScript, Vite, SCSS Modules, React Router, Vitest e Playwright.
- Antes de abrir PR, rode `npm run check` (lint, tipos, testes e build dos dois apps).

Estrutura de pastas

O código compartilhado vive em `src/`; cada app só tem seu HTML, `public/` e ponto de entrada. O alias `@/` aponta para `src/` — use-o em vez de caminhos relativos longos (`../../..`).

```
apps/
  gestor/        index.html + public/ (favicon)
  motorista/     index.html + public/ (manifest, sw.js, ícones)
src/
  entries/       gestor.tsx e motorista.tsx (pontos de entrada)
  apps/motorista/  app do motorista: layouts, pages, services, pwa
  components/    componentes reutilizáveis dos dois apps
  layouts/       casca do gestor (AppLayout, AuthLayout)
  pages/         telas do gestor, uma pasta por módulo
  hooks/         hooks genéricos (useAsync, useMediaQuery…)
  service/       serviços globais (auth)
  styles/        tokens, breakpoints, mixins e estilos globais
  utils/         funções puras (format, masks, classNames)
e2e/             testes Playwright
```

Regra de ouro: código usado por um único app fica dentro dele; código usado pelos dois sobe para `src/components`, `src/hooks` ou `src/utils`. Um app nunca importa de dentro do outro: o gestor não importa de `src/apps/motorista` e o motorista só usa o que é compartilhado (components, hooks, utils, styles, pages/auth e pages/placeholder).

Nomenclatura

Item	Padrão	Exemplo
Componente React	`PascalCase`, arquivo com o mesmo nome	`DriverTable.tsx`
Pasta de componente	`PascalCase`	`components/StatCard/`
Hook	`camelCase` com prefixo `use`	`useDriverFilters.ts`
Serviço	`camelCase` com sufixo `Service`	`driverService.ts`
Estilos	`NomeDoComponente.module.scss`	`Badge.module.scss`
Teste	ao lado do código, `.test.ts(x)`	`driverRules.test.ts`
Tipos de domínio	`PascalCase`, no `types.ts` do módulo	`Driver`, `DriverStatus`
Constantes	`UPPER_SNAKE_CASE`	`MOCK_DELAY_MS`
Classes SCSS	`camelCase`	`.tabActive`
Variáveis SCSS	`$kebab-case`	`$color-primary`

Idioma: código (variáveis, funções, tipos, props) em inglês; textos da interface, rotas e comentários em português do Brasil. Rotas em português e sem acento: `/motoristas`, `/abastecimento`, `/veiculos`.

Booleanos começam com `is`, `has` ou `can` (`isLoading`, `isFullWidth`). **Handlers** usam `handle` no componente que trata o evento (`handleSubmit`) e `on` nas props (`onSubmit`).

Componentes

Cada componente tem **uma pasta** com três arquivos e um `index.ts` que reexporta só o que é público:

```
components/Badge/
  Badge.tsx
  Badge.module.scss
  index.ts        export { Badge } from './Badge'
                  export type { BadgeProps, BadgeVariant } from './Badge'
```

- Importe sempre pelo `index`: `import { Badge } from '@/components/Badge'`.
- Use **funções nomeadas** (`export function Badge`), não `export default` — a exceção são páginas carregadas com `lazy`.
- Exporte a interface de props como `NomeProps`; estenda os atributos nativos quando fizer sentido (`ButtonHTMLAttributes`).
- Props de variação são uniões de strings (`variant: 'primary' | 'secondary'`), não vários booleanos.
- Aceite `className` para o pai ajustar margem e posição; combine classes com `classNames()` de `@/utils/classNames`.
- Componente de `components/` não conhece regra de negócio nem faz requisição. Se precisa de `Driver` ou `Vehicle`, ele pertence à pasta do módulo (`pages/drivers/components/`).
- Antes de criar algo novo, procure em `components/`: `Button`, `Card`, `Badge`, `TextField`, `SelectField`, `CheckboxField`, `Alert`, `StateMessage`, `DataTable`, `Pagination` e `ConfirmDialog` já existem.

Páginas, hooks e serviços

A página só **compõe**: busca os dados por hook, decide o estado da tela e delega o visual a componentes. Lógica fica em hooks e `utils`; acesso a dados fica em serviços.

```
pages/drivers/
  DriversPage.tsx            a tela (+ .module.scss e .test.tsx)
  components/                componentes só deste módulo
  hooks/                     useDrivers, useDriverFilters…
  services/driverService.ts  acesso à API (hoje com mock)
  utils/driverRules.ts       regras puras e testáveis
  types.ts                   tipos do domínio
```

Carregamento de dados. Use `useAsync` e trate sempre os três estados — `loading`, `error` (com botão "Tentar novamente", que chama `reload`) e `success` — usando `StateMessage`. O `loader` passado ao hook precisa ser estável (função de módulo ou `useCallback`).

Serviços. São as únicas funções que falam com o backend. Devolvem os tipos do app (`Driver`), não o formato cru da API; a conversão fica dentro do serviço. Enquanto a API não existe, o serviço devolve dados mockados com a mesma assinatura assíncrona da versão real — integrar é trocar o corpo da função, sem mexer na tela. A URL base vem de `VITE_API_URL` (veja `.env.example`); nunca escreva URLs fixas.

Formatação. Datas, moeda e quilometragem passam por `@/utils/format` (`formatDate`, `formatCurrency`, `formatKm`); não formate à mão dentro do JSX.

Título da aba. Toda tela chama `useDocumentTitle('Nome da tela')` e usa `PageHeader`, que leva o foco para o `<h1>` ao entrar.

Estilos

SCSS Modules, um arquivo por componente. Nada de estilo inline nem classes globais, exceto em `styles/global.scss`.

- **Tokens, nunca valores soltos.** Cores, espaçamentos, raios, sombras e fontes vêm de `styles/_tokens.scss` (`$color-primary`, `$space-3`, `$radius-lg`). Se falta um valor, crie o token — não escreva `#1f5fad` ou `16px` no componente.
- **Mobile-first.** O CSS base é o do celular; telas maiores entram com o mixin `up()` de `_breakpoints.scss` (`sm` 576, `md` 768, `lg` 992, `xl` 1200, `xxl` 1400 px). Nunca use `max-width` em media query.
- **Hover só onde existe mouse:** use o mixin `hover` (já envolve `@media (hover: hover)`), para não "grudar" no toque.
- **Foco visível:** não remova `outline`; o foco global vem do mixin `focus-ring`.
- **Altura da tela:** use `100dvh` (não `100vh`) e `env(safe-area-inset-*)` em barras fixas, por causa do iPhone.

- **Movimento:** transições curtas (`$transition-fast`); o reset global já respeita `prefers-reduced-motion` .
- Importe tokens com `@use '…/styles/tokens' as *;` . Não use `@import` .
- JS não decide layout: prefira CSS. `useMediaQuery` só quando for preciso trocar um componente inteiro.

App do motorista (PWA online-first)

O app do motorista acompanha a rota em tempo real, como um app de corrida. Ele precisa de internet para funcionar; o PWA existe para ser **instalável**, não para trabalhar offline.

- **Nunca faça cache de dados.** O service worker (`apps/motorista/public/sw.js`) só serve `offline.html` quando uma navegação falha. Requisições à API e aos assets sempre vão para a rede. Mudou o `sw.js` ou o `manifest` ? Teste no build de produção (`npm run build:motorista && npm run preview:motorista`) — o SW não é registrado em dev.
- **Rotas na raiz** (`/` , `/viagens` , `/checklist` …), sem prefixo `/motorista` , porque o app tem domínio próprio.
- **Sem conexão:** o `DriverLayout` mostra um aviso fixo via `useOnlineStatus` . Toda ação que envia dados (enviar checklist, registrar abastecimento) deve tratar falha de rede com mensagem clara e permitir tentar de novo; nada é salvo para enviar depois.
- **Mobile-first de verdade:** coluna única de até 32 rem, barra de abas fixa embaixo com 4 itens no máximo, ações principais em botão de largura total ao alcance do polegar, sem tabelas — use listas de cards.
- **Teclado do celular:** use `inputMode` e `autoComplete` corretos nos campos (`decimal` , `numeric` , `tel`).
- **Tela:** respeite `safe-area-inset` (barra superior e inferior) e teste em Pixel 7 (e2e) e num iPhone real quando mexer em barras fixas.
- **Instalação e GPS exigem HTTPS** (ou `localhost`). Peça a permissão de localização no momento em que ela é útil, explicando o motivo, e trate a negação.
- **Bateria e dados:** limite a frequência de envio da posição e pare de enviar quando não houver viagem em andamento.

Acessibilidade e UX

- **Área de toque mínima de 44 px** (`$touch-target` , 2,75 rem) em botões, links, abas e itens de lista.
- **HTML semântico:** `button` para ação, `a` / `Link` para navegação, `nav` com `aria-label` , `main` único por página, um `h1` por tela.

- **Todo campo tem `<label>`** (use `TextField`, `SelectField`, `CheckboxField`); erros aparecem em texto, ligados ao campo (`aria-invalid` + mensagem com `role="alert"`). Cor sozinha nunca comunica estado — acompanhe de texto ou ícone.
- **Estados de tela completos:** carregando, vazio, erro e sucesso, sempre com `StateMessage` ou `Alert`. Erro oferece uma saída ("Tentar novamente").
- **Botão durante envio:** use `isLoading` (desabilita e marca `aria-busy`) para evitar duplo envio.
- **Navegação por teclado** funcionando em tudo, com foco visível; diálogos fecham com `Esc` e devolvem o foco.
- **Ícones decorativos** com `aria-hidden="true"`; ícone sozinho em botão precisa de `aria-label`.
- **Contraste:** use as combinações de cor dos tokens; não crie cinzas novos para texto.

TypeScript e qualidade de código

- **`strict` ligado**, sem `any`. Dado de fora (API, `localStorage`) entra como `unknown` e é validado/convertido no serviço.
- `import type` para tipos (`verbatimModuleSyntax`). Sem imports ou variáveis não usados — o compilador já reprova.
- **Uniões literais no lugar de `enum`:** `const STATUSES = [...] as const` + `type Status = (typeof STATUSES)[number]`, como em `DRIVER_STATUSES`.
- Tipos do backend (`VehicleResponse`) ficam separados dos tipos do app (`Vehicle`); o serviço converte.
- Regras de negócio em funções **puras** em `utils/` (ex.: `driverRules.ts`), fáceis de testar, fora dos componentes.
- Estado: `useState` local primeiro; levante o estado só quando dois irmãos precisarem dele. Nada de biblioteca global sem combinar com o time.
- Efeitos (`useEffect`) só para sincronizar com o mundo externo (eventos, timers, rede). Valor derivado se calcula no render.
- Comentários explicam o **porquê**, não o quê. Código morto e `console.log` não vão para o PR.
- **Formatação e lint são automáticos:** Prettier (`npm run format`) e oxlint (`npm run lint`). Não discuta estilo em review; rode as ferramentas.

Testes

Tipo	Ferramenta	Onde fica	Comando
Unitário e de tela	Vitest + Testing Library	ao lado do código (`*.test.ts(x)`)	`npm test`
Ponta a ponta	Playwright	`e2e/gestor.spec.ts` e `e2e/motorista.spec.ts`	`npm run test:e2e`

- **Regra nova → teste.** Função pura em `utils/` ganha teste unitário; tela nova ganha ao menos um teste de renderização e um do fluxo principal.
- **Teste como o usuário usa:** busque por papel e nome (`getByRole('button', { name: /enviar/i })`), não por classe ou `data-testid`. Use `userEvent`, não `fireEvent`.
- Dados vindos de serviço mockado exigem `findBy*` (assíncrono), não `getBy*`.
- Teste de rota usa `MemoryRouter` com `AppRoutes` (gestor) ou `DriverRoutes` (motorista).
- E2E cobre o caminho feliz de cada app; o do motorista também confere o manifest e o service worker, e roda em viewport de celular (Pixel 7).
- Teste que depende de tempo ou de ordem é bug do teste: conserte, não aumente o timeout à toa.
- Antes do primeiro e2e na máquina: `npx playwright install chromium`.

Fluxo de trabalho

O repositório não traz regras de Git; o que segue é uma **sugestão** para o time ajustar ao que já pratica.

- **Branches curtas** a partir da principal: `feat/solicitacao-de-viagem`, `fix/checklist-envio`, `chore/atualiza-deps`.
- **Commits no imperativo**, uma ideia por commit, com prefixo (`feat:`, `fix:`, `refactor:`, `test:`, `docs:`, `chore:`).
- **PR pequeno** (idealmente até 400 linhas), com o que mudou, por que mudou e como testar. Tela alterada leva print do celular e do desktop.
- Mudou os dois apps? Diga no título (`[gestor]`, `[motorista]` ou `[ambos]`).
- Todo PR precisa de uma aprovação e do `npm run check` verde.

Checklist do PR

- [ ] `npm run check` passa (lint, tipos, testes e build dos dois apps)
- [ ] Componente novo com pasta, `index.ts`, `.module.scss` e props tipadas

- [ ] Estilos só com tokens e mobile-first (`up()`), sem cores ou medidas soltas
- [ ] Estados de carregando, vazio e erro tratados
- [ ] Teste para a regra ou fluxo novo
- [ ] Teclado, foco e área de toque de 44 px verificados
- [ ] Sem `any`, `console.log`, código comentado ou URL fixa
- [ ] App do motorista: nada cacheado, falha de rede tratada e testado em tela de celular